Laboratoire pratique : Construire une API avec Flask : Création de routes, gestion des erreurs et requêtes HTTP

Bienvenue à la partie 2 du laboratoire Flask. Vous travaillerez avec des routes et des requêtes HTTP dans ce laboratoire. Vous vous entraînerez à **créer une petite API RESTful**. Enfin, vous travaillerez avec des **gestionnaires d'erreurs au niveau de l'application** pour des erreurs communes comme :
404 NOT FOUND
500 INTERNAL SERVER ERROR

Vous devriez **connaître tous les concepts dont vous avez besoin pour ce laboratoire** grâce aux vidéos précédentes. N'hésitez pas à interrompre le laboratoire et à revoir le module si vous n'êtes pas sûr de la manière d'effectuer une tâche ou si vous avez besoin de plus d'informations.
En cliquant sur le bouton "Lancer l'application" ci-dessous, vous lancerez l'environnement de laboratoire virtuel SN labs basé sur le cloud avec les instructions pour réaliser ce laboratoire. Votre nom d'utilisateur et votre adresse électronique seront partagés avec SN Labs afin d'authentifier et d'approvisionner votre environnement de laboratoire.
Si vous avez des problèmes avec l'environnement de laboratoire, vous pouvez visualiser le laboratoire en cliquant sur ce lien.

Ce course utilise une application tierce, Laboratoire pratique : Construire une API avec Flask : Création de routes, gestion des erreurs et requêtes HTTP, pour améliorer votre expérience d’apprentissage. L’application référence des informations de base comme votre nom, votre adresse e-mail et votre ID Coursera.

Atelier pratique : Création d'une API avec Flask : Création de routes, Gestion des erreurs et Requêtes HTTP

Temps estimé nécessaire : 45 minutes

Bienvenue dans la partie 2 de l'atelier Flask. Vous **travaillerez avec des routes et des requêtes HTTP** dans cet atelier. Vous **pratiquerez la création d'une petite API RESTful**. Enfin, vous travaillerez avec des **gestionnaires d'erreurs au niveau de l'application** pour des erreurs courantes telles que :

- 404 NOT FOUND
- 500 INTERNAL SERVER ERROR

Vous devriez connaître tous les concepts nécessaires pour cet atelier grâce à l'ensemble précédent de vidéos. N'hésitez pas à mettre l'atelier en pause et à revoir le module si vous n'êtes pas clair sur la façon d'effectuer une tâche ou si vous avez besoin de plus d'informations.

Objectifs d'apprentissage

Après avoir terminé cet atelier, vous serez capable de :

- **Écrire des routes pour traiter les requêtes au serveur Flask à des URL spécifiques**
- Gérer **les paramètres et les arguments envoyés** aux URL
- Écrire des **gestionnaires d'erreurs pour les erreurs de serveur et d'utilisateu**

À propos de Skills Network Cloud IDE

Skills Network Cloud IDE (basé sur Theia et Docker) fournit un environnement pour des laboratoires pratiques liés aux cours et projets. Theia est un IDE (Environnement de Développement Intégré) open source qui fonctionne sur bureau ou dans le cloud. Pour compléter ce laboratoire, vous utiliserez le Cloud IDE basé sur Theia et MongoDB fonctionnant dans un conteneur Docker.

Avis important concernant cet environnement de laboratoire

Veuillez noter que les sessions ne persistent pas dans cet environnement de laboratoire. Chaque fois que vous vous connectez à ce laboratoire, un nouvel environnement est créé pour vous. Toutes les données enregistrées lors de sessions précédentes seront perdues. Prévoyez de compléter ces laboratoires en une seule session pour éviter de perdre vos données.

Configurer l'environnement de laboratoire

Il y a quelques préparations préalables nécessaires avant de commencer le laboratoire.

Ouvrir un terminal

Ouvrez une fenêtre de terminal en utilisant le menu dans l'éditeur : **Terminal** > **Nouveau terminal**.

Dans le terminal, si vous n'êtes pas dans le dossier `/home/project`, changez maintenant pour votre dossier de projet.

```
cd /home/project
```

Copied!Wrap Toggled!Executed!

Créer le répertoire de laboratoire

Vous devriez avoir un répertoire de laboratoire depuis la Partie 1 du laboratoire. Si vous n'avez pas le répertoire, créez-le maintenant.

```
mkdir lab
```

Copied!Wrap Toggled!Executed!

Changez de répertoire pour `lab` :

```
cd lab
```

Copied!Wrap Toggled!Executed!

Vous avez créé un fichier `server.py` dans le répertoire lab à la partie 1 du laboratoire. Créez le fichier s'il n'est pas présent et ajoutez le code de démarrage suivant à celui-ci.

```
# Import the Flask class from the flask module
from flask import Flask

# Create an instance of the Flask class, passing in the name of the current module
app = Flask(__name__)

# Define a route for the root URL ("/")
@app.route("/")
def index():
    # Function that handles requests to the root URL
    # Return a plain text response
    return "hello world"
```

Copied!Wrap Toggled!

> ***Remarque :*** *Assurez-vous que tout le code utilise une indentation de 4 espaces pour éviter* `IndentationError`.

Rappelez-vous que le code ci-dessus crée un serveur Flask et ajoute un point de terminaison d'accueil "/" qui renvoie la chaîne **hello world**. Vous allez maintenant ajouter plus de code à ce fichier dans ce laboratoire.

Pour rappel, utilisez la commande suivante pour exécuter le serveur depuis le terminal :

1. `flask --app server --debug run`

Copied!Wrap Toggled!Executed!

```
Problems        theia@theia-captainfedo1: /home/project/lab ×

[theia: lab]$ flask --app server --debug run
 * Serving Flask app 'server'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WS
GI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 333-842-469
```

Remarque : Si le port `5000` est utilisé, essayez un autre port comme `5001`. Si le serveur ne parvient pas à démarrer, vérifiez `server.py` pour des erreurs de syntaxe ou des imports manquants.

Mes saisies dans le fichier server.py

```
# Import the Flask class from the flask module
from flask import Flask

# Create an instance of the Flask class, passing in the name of the current module
app = Flask(__name__)

# Define a route for the root URL ("/")
@app.route("/")
def index():
    # Function that handles requests to the root URL
    # Return a plain text response
    return "hello world"
```

Mes saisies dans le terminal

```
flask --app server --debug run
```

```
Sortie obtenue
* Serving Flask app 'server'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a
production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 131-913-068
127.0.0.1 - - [03/Jan/2026 10:25:31] "GET / HTTP/1.1" 200 -
 * Detected change in '/home/project/lab/server.py', reloading
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 131-913-068
```

Vous devez maintenant utiliser la commande CURL avec `localhost:5000/`. Notez que le terminal exécute le serveur. Vous pouvez utiliser le bouton `Terminal divisé` pour diviser le terminal et exécuter la commande suivante dans le deuxième onglet.

1. ```
   curl -X GET -i -w '\n' localhost:5000
   ```
   Copied!Wrap Toggled!Executed!

   Mes saisies dans le terminal divisé
   curl -X GET -i -w '\n' localhost:5000

   Sortie obtenue
   HTTP/1.1 200 OK
   Server: Werkzeug/3.1.3 Python/3.11.14
   Date: Sat, 03 Jan 2026 15:25:31 GMT
   Content-Type: text/html; charset=utf-8
   Content-Length: 11
   Connection: close

   hello world
   theia@theia-pierreassam:/home/project$

Optionnel

Si travailler dans le terminal devient difficile en raison de la longueur de l'invite de commande, vous pouvez raccourcir l'invite en utilisant la commande suivante :

1. ```
   export PS1="[\[\033[01;32m\]\u\[\033[00m\]: \[\033[01;34m\]\W\[\033[00m\]]\$ "
   ```
   Copied!Wrap Toggled!Executed!

Étape 1 : Définir le code de statut de réponse

Dans la dernière partie, vous avez vu que Flask envoie automatiquement une réponse réussie `HTTP 200 OK` lorsque vous renvoyez un message. Cependant, vous pouvez également définir explicitement le statut de retour. Rappelez-vous qu'il existe deux façons de le faire, comme discuté dans la vidéo :

1. Renvoie un tuple avec le message
2. Utilisez la méthode **make_response()** pour créer une réponse personnalisée et définir le statut

Vos Tâches

1.

Renvoie un code HTTP personnalisé avec un tuple.

2.

Vous allez réutiliser le fichier `server.py` sur lequel vous avez travaillé dans la dernière partie. Créez une nouvelle méthode nommée `no_content` avec le décorateur `@app.route` et l'URL de `/no_content`. La méthode n'a pas d'arguments. Renvoie un tuple avec le message JSON `No content found`.

```
Mes saisies dans le fichier server.py
# Import the Flask class from the flask module
from flask import Flask

# Create an instance of the Flask class, passing in the name of the current module
app = Flask(__name__)


# Return a tuple with JSON message
@app.route("/no_content")
def no_content():
    return ({"message": "no content found"}, 204)

NB: NE PAS SAISIR CECI DANS RETURN ("no content found", 204)
Si on le fait, le conten type dans la sortie NE SERA PAS application /json mais plutot
Text/html

Ma saisie dans le terminal divisé
curl -X GET -i -w '\n' localhost:5000/no_content

Sortie obtenue
HTTP/1.1 204 NO CONTENT
Server: Werkzeug/3.1.3 Python/3.11.14
Date: Sat, 03 Jan 2026 17:00:10 GMT
Content-Type: application/json
Connection: close
```

3.

Cliquez ici pour un indice.

Vous pouvez utiliser le code de squelette suivant comme point de départ :

```
1. {insert @app decorator}
2. def {insert method name}():
```

```
    """return 'No content found' with a status of 204

    Returns:
        string: No content found
        status code: 204
    """
    return ({insert dictionary here}, {insert HTTP code here})
```

Copied!Wrap Toggled!

Vous pouvez tester le point de terminaison avec la commande CURL suivante :

```
curl -X GET -i -w '\n' localhost:5000/no_content
```

Copied!Wrap Toggled!Executed!

Vous devriez voir une sortie similaire à ce qui suit. Notez le statut de `204` et le type de contenu de `application/json`. Notez que même si vous avez renvoyé un message JSON, il n'est pas renvoyé au client en tant que `204`. Par défaut, rien n'est renvoyé.

```
HTTP/1.1 204 NO CONTENT
Server: Werkzeug/2.2.2 Python/3.7.16
Date: Wed, 28 Dec 2022 19:49:18 GMT
Content-Type: application/json
Connection: close
```

Copied!Wrap Toggled!

Pourquoi utiliser le 204 ?

Le statut 204 No Content est utilisé dans les API REST lorsque une requête réussit mais qu'aucune donnée n'a besoin d'être renvoyée. Par exemple :

- Après avoir supprimé une ressource (par exemple, un utilisateur), le serveur confirme le succès sans envoyer les données supprimées.
- Lors de la confirmation d'une action (par exemple, la mise à jour des paramètres) sans information supplémentaire.
- Ici, nous renvoyons un message JSON. Mais parce que nous indiquons au client via le code de statut 204 qu'aucun contenu ne sera renvoyé, la plupart des clients HTTP ignoreront le corps. Si vous devez afficher un message, utilisez le code de statut 200.

Renvoie un code HTTP personnalisé avec la méthode `make_response()`. La méthode `make_response()` dans Flask est utilisée pour créer manuellement un objet de réponse HTTP complet, vous donnant plus de contrôle sur la réponse que de simplement renvoyer une valeur ou un tuple.

Créez une deuxième méthode nommée `index_explicit` avec le décorateur `@app.route` et une URL de `/exp`. La méthode n'a pas d'arguments. Utilisez la méthode `make_response()` pour créer une nouvelle réponse. Définissez le statut sur 200.

Ma saisie dans le fichier server.py

```
# Import the Flask class from the flask module
from flask import Flask, make_response    NB il faut importer make_response!! pour que ça marche!!

# Create an instance of the Flask class, passing in the name of the current module
app = Flask(__name__)


# Return a personnalize HTTP message
@app.route("/exp")
def exp():
    res = make_response("Hello world")
    res.status_code = 200
```

Ma saisie dans le terminal divisé
curl -X GET -i -w '\n' localhost:5000/exp

Sortie obtenue
HTTP/1.1 200 OK
Server: Werkzeug/3.1.3 Python/3.11.14
Date: Sat, 03 Jan 2026 18:04:08 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 11
Connection: close

Hello world

2.

Cliquez ici pour un indice.

. *Importez la classe Flask du module flask*

Importez la méthode make_response du module flask.

```
1. {insert @app decorator}
2. def {insert method name here}:
3.     """return 'Hello World' message with a status code of 200
4.
5.     Returns:
6.         string: Hello World
7.         status code: 200
8.     """
9.     resp = make_response({insert ditionary here})
10.    resp.status_code = {insert status code here}
11.    return resp
```

Copied!Wrap Toggled!

Vous pouvez tester le point de terminaison avec la commande CURL suivante :

```
curl -X GET -i -w '\n' localhost:5000/exp
```

Copied!Wrap Toggled!Executed!

Vous devriez voir une sortie similaire à celle donnée ci-dessous. Notez le statut de `200`, le type de contenu de `application/json`, et la sortie JSON de `{"message": "Hello World"}` :

```
HTTP/1.1 200 OK
Server: Werkzeug/2.2.2 Python/3.7.16
Date: Wed, 28 Dec 2022 19:55:46 GMT
Content-Type: application/json
Content-Length: 31
Connection: close

{
  "message": "Hello World"
}
```

Copied!Wrap Toggled!

Solution

Vérifiez que votre travail correspond à la solution suivante.

Cliquez ici pour la réponse.

```python
from flask import Flask, make_response

# Create an instance of the Flask class, passing in the name of the current module
app = Flask(__name__)

# Define a route for the root URL ("/")
@app.route("/")
def index():
    # Function that handles requests to the root URL
    # Return a plain text response
    return "hello world"

# Define a route for the "/no_content" URL
@app.route("/no_content")
def no_content():
    """Return 'no content found' with a status of 204.

    Returns:
        tuple: A tuple containing a dictionary and a status code.
    """
    # Create a dictionary with a message and return it with a 204 No Content status code
```

```
    return ({"message": "No content found"}, 204)

# Define a route for the "/exp" URL
@app.route("/exp")
def index_explicit():
    """Return 'Hello World' message with a status code of 200.

    Returns:
        response: A response object containing the message and status code 200.
    """
    # Create a response object with the message "Hello World"
    resp = make_response({"message": "Hello World"})
    # Set the status code of the response to 200
    resp.status_code = 200
    # Return the response object
    return resp
```

Copied!Wrap Toggled!

Étape 2 : Traiter les arguments d'entrée

Il est courant que **les clients passent des arguments dans l'URL**. Vous apprendrez **comment traiter les arguments** dans ce laboratoire. Le laboratoire fournit **une liste de personnes avec leur identifiant, prénom, nom de famille et informations d'adresse dans un objet**. Normalement, ces informations sont **stockées dans une base de données**, mais vous **utilisez une liste codée en dur pour votre cas d'utilisation simple**. Ces données ont été **générées avec Mockaroo**.

Le client enverra des requêtes sous la forme de `http://localhost:5000?q=first_name`. Vous allez créer une méthode qui acceptera un prénom comme entrée et renverra une personne avec ce prénom.

Cliquez ici pour copier les données dans le fichier.

Copiez la liste suivante dans le fichier `server.py` :

```
from flask import Flask, make_response
app = Flask(__name__)

data = [
    {
        "id": "3b58aade-8415-49dd-88db-8d7bce14932a",
        "first_name": "Tanya",
        "last_name": "Slad",
        "graduation_year": 1996,
        "address": "043 Heath Hill",
        "city": "Dayton",
        "zip": "45426",
        "country": "United States",
        "avatar": "http://dummyimage.com/139x100.png/cc0000/ffffff",
    },
    {
        "id": "d64efd92-ca8e-40da-b234-47e6403eb167",
        "first_name": "Ferdy",
        "last_name": "Garrow",
        "graduation_year": 1970,
        "address": "10 Wayridge Terrace",
        "city": "North Little Rock",
        "zip": "72199",
        "country": "United States",
        "avatar": "http://dummyimage.com/148x100.png/dddddd/000000",
    },
    {
        "id": "66c09925-589a-43b6-9a5d-d1601cf53287",
        "first_name": "Lilla",
        "last_name": "Aupol",
        "graduation_year": 1985,
        "address": "637 Carey Pass",
```

```
        "city": "Gainesville",
        "zip": "32627",
        "country": "United States",
        "avatar": "http://dummyimage.com/174x100.png/ff4444/ffffff",
    },
    {
        "id": "0dd63e57-0b5f-44bc-94ae-5c1b4947cb49",
        "first_name": "Abdel",
        "last_name": "Duke",
        "graduation_year": 1995,
        "address": "2 Lake View Point",
        "city": "Shreveport",
        "zip": "71105",
        "country": "United States",
        "avatar": "http://dummyimage.com/145x100.png/dddddd/000000",
    },
    {
        "id": "a3d8adba-4c20-495f-b4c4-f7de8b9cfb15",
        "first_name": "Corby",
        "last_name": "Tettley",
        "graduation_year": 1984,
        "address": "90329 Amoth Drive",
        "city": "Boulder",
        "zip": "80305",
        "country": "United States",
        "avatar": "http://dummyimage.com/198x100.png/cc0000/ffffff",
    }
]
```

Copied!Wrap Toggled!

Confirons que les données ont été copiées dans le fichier. Copiez le code suivant dans le fichier **server.py** pour créer un point de terminaison qui renvoie les données de la personne au client au format JSON.

```
@app.route("/data")
def get_data():
    try:
        # Check if 'data' exists and has a length greater than 0
        if data and len(data) > 0:
            # Return a JSON response with a message indicating the length of the data
            return {"message": f"Data of length {len(data)} found"}
        else:
            # If 'data' is empty, return a JSON response with a 500 Internal Server Error status code
```

```
            return {"message": "Data is empty"}, 500
    except NameError:
        # Handle the case where 'data' is not defined
        # Return a JSON response with a 404 Not Found status code
        return {"message": "Data not found"}, 404
```

Copied!Wrap Toggled!

Le code ci-dessus vérifie simplement si la variable `data` existe. Si ce n'est pas le cas, une `NameError` est levée et un `HTTP 404` est retourné. Si `data` existe et est vide, un `HTTP 500` est retourné. Si `data` existe et n'est pas vide, un message de succès `HTTP 200` est retourné.

Exécutez une commande CURL pour confirmer que vous recevez le message de succès :

```
curl -X GET -i -w '\n' localhost:5000/data
```

Copied!Wrap Toggled!Executed!

Résultat attendu :

```
HTTP/1.1 200 OK
Server: Werkzeug/2.2.2 Python/3.7.16
Date: Wed, 28 Dec 2022 20:51:56 GMT
Content-Type: application/json
Content-Length: 42
Connection: close

{
  "message": "Data of length 5 found"
}
```

Copied!Wrap Toggled!

Vos Tâches

Créez une méthode appelée `name_search` avec le décorateur `@app.route`. Cette méthode doit être appelée lorsqu'un client demande l'URL `/name_search`. La méthode n'acceptera aucun paramètre, mais cherchera l'argument `q` dans l'URL de la requête entrante. Cet argument contient le prénom que le client recherche. La méthode retourne :

- Les informations de la personne avec un statut de `HTTP 200` si le prénom est trouvé dans les données
- Un message `Paramètre d'entrée invalide` avec un statut de `HTTP 400` si l'argument `q` est manquant dans la requête
- Un message `Personne non trouvée` avec un code de statut `HTTP 404` si la personne n'est pas trouvée dans les données

Indice

Assurez-vous d'importer le module `request` de Flask. Vous l'utiliserez pour obtenir le prénom de la requête HTTP.

```
from flask import request
```

Copied!Wrap Toggled!

Vous pouvez utiliser le code suivant comme point de départ :

Cliquez ici pour un indice.

```
@app.route("/name_search")
def name_search():
    """Find a person in the database.

    Returns:
        json: Person if found, with status of 200
        404: If not found
        400: If argument 'q' is missing
        422: If argument 'q' is present but invalid
    """
    # Get the argument 'q' from the query parameters of the request
    query = request.args.get('q')

    # Check if the query parameter 'q' is missing
    if query is None:
        return {"message": "Query parameter 'q' is missing"}, 400

    # Check if the query parameter is present but invalid (e.g., empty or numeric)
    if query.strip() == "" or query.isdigit():
        return {"message": "Invalid input parameter"}, 422

    # Iterate through the 'data' list to look for the person whose first name matches the query
    for person in data:
        if query.lower() in person["first_name"].lower():
            # If a match is found, return the person as a JSON response with a 200 OK status code
            return person, 200

    # If no match is found, return a JSON response with a message indicating the person was not found and a 404 Not Found status code
    return {"message": "Person not found"}, 404
```

Copied!Wrap Toggled!

Vous pouvez tester le point de terminaison avec la commande CURL suivante. Assurez-vous que le serveur fonctionne dans le terminal comme dans les étapes précédentes.

```
curl -X GET -i -w '\n' "localhost:5000/name_search?q=Abdel"
```

```
Copied!Wrap Toggled!Executed!
```

Vous devriez voir une sortie similaire à celle donnée ci-dessous. Notez le statut de `200`, le Content-Type de `application/json`, et la sortie JSON de la personne avec le prénom **Abdel** :

```
HTTP/1.1 200 OK
Server: Werkzeug/2.2.2 Python/3.7.16
Date: Wed, 28 Dec 2022 21:14:31 GMT
Content-Type: application/json
Content-Length: 295
Connection: close

{
  "address": "2 Lake View Point",
  "avatar": "http://dummyimage.com/145x100.png/dddddd/000000",
  "city": "Shreveport",
  "country": "United States",
  "first_name": "Abdel",
  "graduation_year": 1995,
  "id": "0dd63e57-0b5f-44bc-94ae-5c1b4947cb49",
  "last_name": "Duke",
  "zip": "71105"
}
```

Mes saisies dans le fichier server.py

```
from flask import Flask, make_response
app = Flask(__name__)

data = [
    {
        "id": "3b58aade-8415-49dd-88db-8d7bce14932a",
        "first_name": "Tanya",
        "last_name": "Slad",
        "graduation_year": 1996,
        "address": "043 Heath Hill",
        "city": "Dayton",
        "zip": "45426",
        "country": "United States",
        "avatar": "http://dummyimage.com/139x100.png/cc0000/ffffff",
    },
    {
        "id": "d64efd92-ca8e-40da-b234-47e6403eb167",
        "first_name": "Ferdy",
        "last_name": "Garrow",
        "graduation_year": 1970,
        "address": "10 Wayridge Terrace",
        "city": "North Little Rock",
        "zip": "72199",
```

```
        "country": "United States",
        "avatar": "http://dummyimage.com/148x100.png/dddddd/000000",
    },
    {
        "id": "66c09925-589a-43b6-9a5d-d1601cf53287",
        "first_name": "Lilla",
        "last_name": "Aupol",
        "graduation_year": 1985,
        "address": "637 Carey Pass",
        "city": "Gainesville",
        "zip": "32627",
        "country": "United States",
        "avatar": "http://dummyimage.com/174x100.png/ff4444/ffffff",
    },
    {
        "id": "0dd63e57-0b5f-44bc-94ae-5c1b4947cb49",
        "first_name": "Abdel",
        "last_name": "Duke",
        "graduation_year": 1995,
        "address": "2 Lake View Point",
        "city": "Shreveport",
        "zip": "71105",
        "country": "United States",
        "avatar": "http://dummyimage.com/145x100.png/dddddd/000000",
    },
    {
        "id": "a3d8adba-4c20-495f-b4c4-f7de8b9cfb15",
        "first_name": "Corby",
        "last_name": "Tettley",
        "graduation_year": 1984,
        "address": "90329 Amoth Drive",
        "city": "Boulder",
        "zip": "80305",
        "country": "United States",
        "avatar": "http://dummyimage.com/198x100.png/cc0000/ffffff",
    }
]
```


```
@app.route("/data")
def get_data():
    try:
        # Check if 'data' exists and has a length greater than 0
        if data and len(data) > 0:
            # Return a JSON response with a message indicating the length of the data
            return {"message": f"Data of length {len(data)} found"}
        else:
            # If 'data' is empty, return a JSON response with a 500 Internal Server Error
status code
            return {"message": "Data is empty"}, 500
    except NameError:
        # Handle the case where 'data' is not defined
        # Return a JSON response with a 404 Not Found status code
        return {"message": "Data not found"}, 404
```

```python
from flask import request
@app.route("/name_search")
def name_search():
    query = request.args.get("q")

    if query is None:
        return {"error message": "paramètre 'q' manquant"}, 400

    # Check if the query parameter is present but invalid (e.g., empty or numeric)
    if query.strip() == "" or query.isdigit():
        return {"message": "Invalid input parameter"}, 422

    # Iterate through the 'data' list to look for the person whose first name matches the query
    for person in data:
        if query.lower() in person["first_name"].lower():
            # If a match is found, return the person as a JSON response with a 200 OK status code
            return person, 200

    # If no match is found, return a JSON response with a message indicating the person was not found and a 404 Not Found status code
    return {"message": "Person not found"}, 404
```

```
Ma saisie dans le terminal divisé
curl -X GET -i -w '\n' "localhost:5000/name_search?q=Abdel"


Sortie obtenue
HTTP/1.1 200 OK
Server: Werkzeug/3.1.3 Python/3.11.14
Date: Sat, 03 Jan 2026 19:53:23 GMT
Content-Type: application/json
Content-Length: 295
Connection: close

{
  "address": "2 Lake View Point",
  "avatar": "http://dummyimage.com/145x100.png/dddddd/000000",
  "city": "Shreveport",
  "country": "United States",
  "first_name": "Abdel",
  "graduation_year": 1995,
  "id": "0dd63e57-0b5f-44bc-94ae-5c1b4947cb49",
  "last_name": "Duke",
  "zip": "71105"
}
```

Copied!Wrap Toggled!

Ensuite, testez que la méthode renvoie `HTTP 422` si l'argument `q` est invalide :

1. `curl -X GET -i -w '\n' "localhost:5000/name_search?q=123"`

Copied!Wrap Toggled!Executed!

ou

```
curl -X GET -i -w '\n' "localhost:5000/name_search?q="
```

Copied!Wrap Toggled!Executed!

> *Remarque : Utilisez `HTTP 400` (Mauvaise requête) lorsque le paramètre de requête requis est manquant. Utilisez `HTTP 422` (Entité non traitable) uniquement si le paramètre est présent mais a un contenu invalide ou inacceptable.*

Vous devriez voir une sortie similaire à celle donnée ci-dessous. Notez le statut de `422`, le type de contenu de `application/json`, et la sortie JSON de `Paramètre d'entrée invalide` :

```
HTTP/1.1 422 UNPROCESSABLE ENTITY
Server: Werkzeug/2.2.2 Python/3.7.16
Date: Wed, 28 Dec 2022 21:16:07 GMT
Content-Type: application/json
Content-Length: 43
Connection: close

{
  "message": "Invalid input parameter"
}
```

Copied!Wrap Toggled!

Enfin, testons le cas où le prénom n'est pas présent dans notre liste de personnes :

```
curl -X GET -i -w '\n' "localhost:5000/name_search?q=qwerty"
```

Copied!Wrap Toggled!Executed!

Vous devriez voir une sortie similaire à celle donnée ci-dessous. Notez le statut de `404`, le type de contenu de `application/json`, et la sortie JSON de `Person not found` :

```
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/2.2.2 Python/3.7.16
Date: Wed, 28 Dec 2022 21:17:28 GMT
Content-Type: application/json
Content-Length: 36
Connection: close

{
  "message": "Person not found"
}
```

Copied!Wrap Toggled!

Solution

Vérifiez que votre travail correspond à la solution suivante. Il existe d'autres façons de mettre en œuvre cette solution également.

Cliquez ici pour la réponse.

```
1. @app.route("/name_search")
2. def name_search():
3.     """Find a person in the database based on the provided query parameter.
4.
5.     Returns:
6.         json: Person if found, with status of 200
7.         404: If not found
8.         400: If the argument 'q' is missing
9.         422: If the argument 'q' is present but invalid (e.g., empty or numeric)
10.    """
11.    # Get the 'q' query parameter from the request URL
12.    query = request.args.get("q")
13.
14.    # Check if the query parameter 'q' is missing
15.    if query is None:
16.        return {"message": "Query parameter 'q' is missing"}, 400
17.
18.    # Check if the query parameter is present but invalid (empty or numeric)
19.    if query.strip() == "" or query.isdigit():
20.        return {"message": "Invalid input parameter"}, 422
21.
22.    # Iterate through the 'data' list to search for a matching person
23.    for person in data:
24.        # Check if the query string is present in the person's first name (case-insensitive)
25.        if query.lower() in person["first_name"].lower():
26.            # Return the matching person as a JSON response with a 200 OK status code
27.            return person, 200
28.
29. # If no matching person is found, return a JSON response with a message and a 404 Not Found
30.    return {"message": "Person not found"}, 404
```

Copied!Wrap Toggled!

Étape 3 : Ajouter des URL dynamiques

Une partie importante de la **programmation back-end consiste à créer des API**. Une **API est un contrat entre un fournisseur et un utilisateur.** Il est courant de créer des **API RESTful qui peuvent être appelées par le front-end ou d'autres clients**. Dans u**ne API basée sur REST, les informations sur la ressource sont envoyées dans l'URL de la requête**. Par exemple, avec votre ressource ou personnes, le client peut envoyer la requête suivante :

1. `GET http://localhost/person/unique_identifier`

Copied!Wrap Toggled!

Cette demande concerne une personne avec un identifiant unique. Un autre exemple est :

1. `DELETE http://localhost/person/unique_identifier`

Copied!Wrap Toggled!

Dans ce cas, le client demande de supprimer la personne avec cet identifiant unique.

Vos Tâches

Vous devez implémenter ces deux points de terminaison dans cet exercice. Vous allez également implémenter une méthode `count` qui retourne le nombre total de personnes dans la liste `data`. Cela aidera à confirmer que les deux méthodes GET et DELETE fonctionnent comme prévu.

Tâche 1 : Créer le point de terminaison GET /count

Créez le point de terminaison **/count**.

Ajoutez le décorateur `@app.get()` pour l'URL `/count`. Définissez la fonction count qui retourne simplement le nombre d'éléments dans la liste `data`.

1. Cliquez ici pour un indice.
   Utilisez la méthode ***len()*** *pour retourner le nombre d' éléments dans la liste* ***data.***
2.
3.

```
i.   @app.route("/count")
ii.  def count():
iii.     try:
iv.          # Tentez de retourner une réponse JSON avec le nombre d'éléments dans 'data'
v.           # Remplacez {insérer le code pour trouver la longueur de data} par len(data) pour
     obtenir la longueur de la collection 'data'
vi.          return {"data count": len(data)}, 200
vii.     except NameError:
viii.        # Si 'data' n'est pas défini et soulève une NameError
ix.          # Retournez une réponse JSON avec un message et un code de statut 500 Internal Server
     Error
x.           return {"message": "data not defined"}, 500
```

Copied!Wrap Toggled!

Testez la méthode **count** en appelant le point de terminaison.

```
curl -X GET -i -w '\n' "localhost:5000/count"
```

Copied!Wrap Toggled!Executed!

Vous devriez voir une sortie avec le nombre d'éléments dans la liste des données.

```
HTTP/1.1 200 OK
Server: Werkzeug/2.2.2 Python/3.7.16
Date: Sam, 31 Déc 2022 22:41:35 GMT
Content-Type: application/json
Content-Length: 22
Connection: close

{
  "data count": 5
}
```

```
Mes saisies dans le fichier server.py
from flask import Flask, make_response
app = Flask(__name__)

data = [
    {
        "id": "3b58aade-8415-49dd-88db-8d7bce14932a",
        "first_name": "Tanya",
        "last_name": "Slad",
        "graduation_year": 1996,
        "address": "043 Heath Hill",
        "city": "Dayton",
        "zip": "45426",
        "country": "United States",
        "avatar": "http://dummyimage.com/139x100.png/cc0000/ffffff",
    },
    {
        "id": "d64efd92-ca8e-40da-b234-47e6403eb167",
        "first_name": "Ferdy",
        "last_name": "Garrow",
        "graduation_year": 1970,
        "address": "10 Wayridge Terrace",
        "city": "North Little Rock",
        "zip": "72199",
        "country": "United States",
        "avatar": "http://dummyimage.com/148x100.png/dddddd/000000",
    },
    {
        "id": "66c09925-589a-43b6-9a5d-d1601cf53287",
        "first_name": "Lilla",
        "last_name": "Aupol",
        "graduation_year": 1985,
        "address": "637 Carey Pass",
        "city": "Gainesville",
```

```
        "zip": "32627",
        "country": "United States",
        "avatar": "http://dummyimage.com/174x100.png/ff4444/ffffff",
    },
    {
        "id": "0dd63e57-0b5f-44bc-94ae-5c1b4947cb49",
        "first_name": "Abdel",
        "last_name": "Duke",
        "graduation_year": 1995,
        "address": "2 Lake View Point",
        "city": "Shreveport",
        "zip": "71105",
        "country": "United States",
        "avatar": "http://dummyimage.com/145x100.png/dddddd/000000",
    },
    {
        "id": "a3d8adba-4c20-495f-b4c4-f7de8b9cfb15",
        "first_name": "Corby",
        "last_name": "Tettley",
        "graduation_year": 1984,
        "address": "90329 Amoth Drive",
        "city": "Boulder",
        "zip": "80305",
        "country": "United States",
        "avatar": "http://dummyimage.com/198x100.png/cc0000/ffffff",
    }
]

@app.route("/data")
def get_data():
    try:
        # Check if 'data' exists and has a length greater than 0
        if data and len(data) > 0:
            # Return a JSON response with a message indicating the length of the data
            return {"message": f"Data of length {len(data)} found"}
        else:
            # If 'data' is empty, return a JSON response with a 500 Internal Server Error
status code
            return {"message": "Data is empty"}, 500
    except NameError:
        # Handle the case where 'data' is not defined
        # Return a JSON response with a 404 Not Found status code
        return {"message": "Data not found"}, 404


from flask import request
@app.route("/name_search")
def name_search():
    query = request.args.get("q")

    if query is None:
        return {"error message": "paramètre 'q' manquant"}, 400

    # Check if the query parameter is present but invalid (e.g., empty or numeric)
    if query.strip() == "" or query.isdigit():
```

```
        return {"message": "Invalid input parameter"}, 422

    # Iterate through the 'data' list to look for the person whose first name matches the query
    for person in data:
        if query.lower() in person["first_name"].lower():
            # If a match is found, return the person as a JSON response with a 200 OK status code
            return person, 200

    # If no match is found, return a JSON response with a message indicating the person was not found and a 404 Not Found status code
    return {"message": "Person not found"}, 404

@app.route("/count")
def count():
    try:
        # Tentez de retourner une réponse JSON avec le nombre d'éléments dans 'data'
        # Remplacez {insérer le code pour trouver la longueur de data} par len(data) pour obtenir la longueur de la collection 'data'
        return {"data_count": len(data)}, 200
    except NameError:
        # Si 'data' n'est pas défini et soulève une NameError
        # Retournez une réponse JSON avec un message et un code de statut 500 Internal Server Error
        return {"message": "data not defined"}, 500
```

```
Ma saisie dans le terminal divisé
curl -X GET -i -w '\n' "localhost:5000/count"

Sortie obtenue
HTTP/1.1 200 OK
Server: Werkzeug/3.1.3 Python/3.11.14
Date: Sat, 03 Jan 2026 20:38:50 GMT
Content-Type: application/json
Content-Length: 22
Connection: close

{
  "data_count": 5
}
```

Copied!Wrap Toggled!

Tâche 2 : Créer le point de terminaison GET /person/id

1.

Implémentez le point de terminaison **GET** pour demander une personne par id.

2.

Créez un nouveau point de terminaison pour `http://localhost/person/unique_identifier`. La méthode devrait s'appeler `find_by_uuid`. Elle devrait prendre un argument de type UUID et retourner la personne en JSON

si trouvée. Si la personne n'est pas trouvée, la méthode devrait retourner un 404 avec un message de **person not found**. Enfin, le client (curl) ne devrait pouvoir appeler cette méthode qu'en passant un id de type UUID valide.

3. Cliquez ici pour un indice.
   - *utilisez la syntaxe type:name pour ne permettre qu' aux appelants de passer un UUID valide.*
   - *comparer uuid avec une chaîne retourne False. Assurez-vous de convertir UUID en str lors de la comparaison avec l' attribut id de la personne.*

```
@app.route("/person/<var_name>")
def find_by_uuid(var_name):
    # Itérez à travers la liste 'data' pour rechercher une personne avec un ID correspondant
    for person in data:
        # Vérifiez si le champ 'id' de la personne correspond au paramètre 'var_name'
        if person["id"] == str(var_name):
            # Retournez la personne en tant que réponse JSON si une correspondance est trouvée
            return person

    # Retournez une réponse JSON avec un message et un code de statut 404 Not Found si aucune personne correspondante n'est trouvée
    return {"message": "Person not found"}, 404
```

Copied!Wrap Toggled!Executed!

Testez l'URL **/person/uuid** en appelant le point de terminaison.

```
curl -X GET -i localhost:5000/person/66c09925-589a-43b6-9a5d-d1601cf53287
```

Copied!Wrap Toggled!Executed!

Vous devriez voir une sortie avec la personne et un code HTTP de 200.

```
HTTP/1.1 200 OK
Server: Werkzeug/2.2.2 Python/3.7.16
Date: Sam, 31 Déc 2022 22:48:32 GMT
Content-Type: application/json
Content-Length: 294
Connection: close

{
  "address": "637 Carey Pass",
  "avatar": "http://dummyimage.com/174x100.png/ff4444/ffffff",
  "city": "Gainesville",
  "country": "United States",
  "first_name": "Lilla",
  "graduation_year": 1985,
  "id": "66c09925-589a-43b6-9a5d-d1601cf53287",
```

vi. "last_name": "Aupol",

vii. "zip": "32627"

viii. }

Saisies dans le fichier server.py

```
from flask import Flask, make_response
app = Flask(__name__)

data = [
    {
        "id": "3b58aade-8415-49dd-88db-8d7bce14932a",
        "first_name": "Tanya",
        "last_name": "Slad",
        "graduation_year": 1996,
        "address": "043 Heath Hill",
        "city": "Dayton",
        "zip": "45426",
        "country": "United States",
        "avatar": "http://dummyimage.com/139x100.png/cc0000/ffffff",
    },
    {
        "id": "d64efd92-ca8e-40da-b234-47e6403eb167",
        "first_name": "Ferdy",
        "last_name": "Garrow",
        "graduation_year": 1970,
        "address": "10 Wayridge Terrace",
        "city": "North Little Rock",
        "zip": "72199",
        "country": "United States",
        "avatar": "http://dummyimage.com/148x100.png/dddddd/000000",
    },
    {
        "id": "66c09925-589a-43b6-9a5d-d1601cf53287",
        "first_name": "Lilla",
        "last_name": "Aupol",
        "graduation_year": 1985,
        "address": "637 Carey Pass",
        "city": "Gainesville",
        "zip": "32627",
        "country": "United States",
        "avatar": "http://dummyimage.com/174x100.png/ff4444/ffffff",
    },
    {
        "id": "0dd63e57-0b5f-44bc-94ae-5c1b4947cb49",
        "first_name": "Abdel",
        "last_name": "Duke",
        "graduation_year": 1995,
        "address": "2 Lake View Point",
        "city": "Shreveport",
        "zip": "71105",
        "country": "United States",
        "avatar": "http://dummyimage.com/145x100.png/dddddd/000000",
    },
    {
        "id": "a3d8adba-4c20-495f-b4c4-f7de8b9cfb15",
```

```
        "first_name": "Corby",
        "last_name": "Tettley",
        "graduation_year": 1984,
        "address": "90329 Amoth Drive",
        "city": "Boulder",
        "zip": "80305",
        "country": "United States",
        "avatar": "http://dummyimage.com/198x100.png/cc0000/ffffff",
    }
]

@app.route("/data")
def get_data():
    try:
        # Check if 'data' exists and has a length greater than 0
        if data and len(data) > 0:
            # Return a JSON response with a message indicating the length of the data
            return {"message": f"Data of length {len(data)} found"}
        else:
            # If 'data' is empty, return a JSON response with a 500 Internal Server Error
status code
            return {"message": "Data is empty"}, 500
    except NameError:
        # Handle the case where 'data' is not defined
        # Return a JSON response with a 404 Not Found status code
        return {"message": "Data not found"}, 404


from flask import request
@app.route("/name_search")
def name_search():
    query = request.args.get("q")

    if query is None:
        return {"error message": "paramètre 'q' manquant"}, 400

    # Check if the query parameter is present but invalid (e.g., empty or numeric)
    if query.strip() == "" or query.isdigit():
        return {"message": "Invalid input parameter"}, 422

    # Iterate through the 'data' list to look for the person whose first name matches the
query
    for person in data:
        if query.lower() in person["first_name"].lower():
            # If a match is found, return the person as a JSON response with a 200 OK status
code
            return person, 200

    # If no match is found, return a JSON response with a message indicating the person was
not found and a 404 Not Found status code
    return {"message": "Person not found"}, 404

@app.route("/count")
def count():
    try:
```

```
        # Tentez de retourner une réponse JSON avec le nombre d'éléments dans 'data'
        # Remplacez {insérer le code pour trouver la longueur de data} par len(data) pour
obtenir la longueur de la collection 'data'
        return {"data_count": len(data)}, 200
    except NameError:
        # Si 'data' n'est pas défini et soulève une NameError
        # Retournez une réponse JSON avec un message et un code de statut 500 Internal Server
Error
        return {"message": "data not defined"}, 500

@app.route("/person/<var_name>")
def find_by_uuid(var_name):
    # Itérez à travers la liste 'data' pour rechercher une personne avec un ID correspondant
    for person in data:
        # Vérifiez si le champ 'id' de la personne correspond au paramètre 'var_name'
        if person["id"] == str(var_name):
            # Retournez la personne en tant que réponse JSON si une correspondance est
trouvée
            return person

    # Retournez une réponse JSON avec un message et un code de statut 404 Not Found si aucune
personne correspondante n'est trouvée
    return {"message": "Person not found"}, 404
```

```
Saisies dans le terminal divisé
curl -X GET -i localhost:5000/person/66c09925-589a-43b6-9a5d-d1601cf53287

Sortie obtenue
HTTP/1.1 200 OK
Server: Werkzeug/3.1.3 Python/3.11.14
Date: Sat, 03 Jan 2026 20:57:03 GMT
Content-Type: application/json
Content-Length: 294
Connection: close

{
  "address": "637 Carey Pass",
  "avatar": "http://dummyimage.com/174x100.png/ff4444/ffffff",
  "city": "Gainesville",
  "country": "United States",
  "first_name": "Lilla",
  "graduation_year": 1985,
  "id": "66c09925-589a-43b6-9a5d-d1601cf53287",
  "last_name": "Aupol",
  "zip": "32627"
}
```

Copied!Wrap Toggled!

Si vous passez un UUID invalide, le serveur devrait retourner un message 404.

i.
```
curl -X GET -i localhost:5000/person/not-a-valid-uuid
```

Copied!Wrap Toggled!Executed!

Vous devriez voir une erreur dans la sortie avec un code de 404. Flask retourne automatiquement du HTML, vous changerez le HTML dans la prochaine partie du laboratoire pour retourner du JSON par défaut sur toutes les erreurs, y compris 404.

```
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/2.2.2 Python/3.7.16
Date: Sam, 31 Déc 2022 22:50:52 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 207
Connection: close

<!doctype html>
<html lang=en>
<title>404 Not Found</title>
<h1>Not Found</h1>
<p>L'URL demandée n'a pas été trouvée sur le serveur. Si vous avez saisi l'URL manuellement, veuillez vérifier votre orthographe et réessayer.</p>
```

Copied!Wrap Toggled!

Enfin, passez un UUID valide qui n'existe pas dans la liste des données. La méthode devrait retourner un 404 avec un message de **person not found**.

```
curl -X GET -i localhost:5000/person/11111111-589a-43b6-9a5d-d1601cf51111
```

Copied!Wrap Toggled!Executed!

Vous devriez voir une réponse JSON avec un code HTTP de 404 et un message de **person not found**.

```
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/2.2.2 Python/3.7.16
Date: Sam, 31 Déc 2022 22:52:24 GMT
Content-Type: application/json
Content-Length: 36
Connection: close

{
  "message": "person not found"
}
```

Copied!Wrap Toggled!

Tâche 3 : Créer le point de terminaison DELETE /person/id

1.

Implémentez le point de terminaison **DELETE** pour supprimer une ressource personne.

2.

Créez un nouveau point de terminaison pour DELETE `http://localhost/person/unique_identifier`. La méthode devrait s'appeler `delete_by_uuid`. Elle devrait prendre un argument de type UUID et supprimer la personne de la liste **data** avec cet id. Si la personne n'est pas trouvée, la méthode devrait retourner un 404 avec un message de **person not found**. Enfin, le client (curl) devrait appeler cette méthode en passant un id de type UUID valide.

3. Cliquez ici pour un indice.
   - *utilisez la syntaxe type:name pour ne permettre qu' aux appelants de passer un UUID valide.*
   - *comparer uuid avec une chaîne retourne False. Assurez-vous de convertir UUID en str lors de la comparaison avec l' attribut id de la personne.*
   - *passez le type de méthode* ***DELETE*** *comme deuxième argument au décorateur @app ou utilisez le décorateur @app.delete().*

```
@app.route("/person/<uuid:id>", methods=['DELETE'])
def delete_person(var_name): c'est une erreur voir plus bas (mes saisies)
    for person in data:
        if person["id"] == str(var_name):
            # Supprimez la personne de la liste des données
            data.remove(person)
            # Retournez une réponse JSON avec un message et un code de statut HTTP 200 (OK)
            return {"message": "Person with ID deleted"}, 200
    # Si aucune personne avec l'ID donné n'est trouvée, retournez une réponse JSON avec un message et un code de statut HTTP 404 (Not Found)
    return {"message": "Person not found"}, 404
```

Copied!Wrap Toggled!Executed!

Testez l'URL DELETE **/person/uuid** en appelant le point de terminaison.

```
curl -X DELETE -i localhost:5000/person/66c09925-589a-43b6-9a5d-d1601cf53287
```

Copied!Wrap Toggled!Executed!

Vous devriez voir une sortie avec l'id de la personne supprimée et un code de statut de 200.

```
HTTP/1.1 200 OK
Server: Werkzeug/2.2.2 Python/3.7.16
Date: Sam, 31 Déc 2022 23:00:17 GMT
Content-Type: application/json
Content-Length: 56
Connection: close

{
  "message": "Person with ID 66c09925-589a-43b6-9a5d-d1601cf53287 deleted"
}
```

```
Mes saisie dans le fichier server.py
@app.route("/person/<uuid:id>", methods=['DELETE'])
def delete_by_uuid(id):
    # Itérez à travers la liste 'data' pour rechercher une personne avec un ID correspondant
    for person in data:
        # Vérifiez si le champ 'id' de la personne correspond au paramètre 'id'
        if person["id"] == str(id):
            # Supprimez la personne de la liste 'data'
            data.remove(person)
            # Retournez une réponse JSON avec un message confirmant la suppression et un code
de statut 200 OK
            return {"message": f"Person with ID {id} deleted"}, 200
    # Si aucune personne correspondante n'est trouvée, retournez une réponse JSON avec un
message et un code de statut 404 Not Found
    return {"message": "person not found"}, 404
```

```
Saisie dans le terminal partagé
curl -X DELETE -i localhost:5000/person/66c09925-589a-43b6-9a5d-d1601cf53287


Sortie obtenue
HTTP/1.1 200 OK
Server: Werkzeug/3.1.3 Python/3.11.14
Date: Sat, 03 Jan 2026 21:21:43 GMT
Content-Type: application/json
Content-Length: 79
Connection: close

{
  "message": "Person with ID 66c09925-589a-43b6-9a5d-d1601cf53287 deleted"
}
```

Copied!Wrap Toggled!

Vous pouvez maintenant utiliser le point de terminaison **count** que vous avez ajouté précédemment pour tester si le nombre de personnes a diminué d'un.

```
i.  curl -X GET -i localhost:5000/count
```

Copied!Wrap Toggled!Executed!

Vous devriez voir le compte retourné réduit de un.

```
i.   HTTP/1.1 200 OK
ii.  Server: Werkzeug/2.2.2 Python/3.7.16
iii. Date: Sam, 31 Déc 2022 23:06:55 GMT
iv.  Content-Type: application/json
v.   Content-Length: 22
vi.  Connection: close
vii.
viii. {
ix.   "data count": 4
```

```
x.  }
```

Copied!Wrap Toggled!

Si vous passez un UUID invalide, le serveur devrait retourner un message 404.

```
i.  curl -X DELETE -i localhost:5000/person/not-a-valid-uuid
```

Copied!Wrap Toggled!Executed!

Vous devriez voir une erreur dans la sortie avec un code de 404. Flask retourne automatiquement du HTML, et nous allons changer le HTML dans la prochaine partie du laboratoire pour retourner du JSON par défaut sur toutes les erreurs, y compris 404.

```
i.   HTTP/1.1 404 NOT FOUND
ii.  Server: Werkzeug/2.2.2 Python/3.7.16
iii. Date: Sam, 31 Déc 2022 23:05:09 GMT
iv.  Content-Type: text/html; charset=utf-8
v.   Content-Length: 207
vi.  Connection: close
vii.
viii. <!doctype html>
ix.  <html lang=en>
x.   <title>404 Not Found</title>
xi.  <h1>Not Found</h1>
xii. <p>L'URL demandée n'a pas été trouvée sur le serveur. Si vous avez saisi l'URL manuellement,
     veuillez vérifier votre orthographe et réessayer.</p>
```

Copied!Wrap Toggled!

Enfin, passez un UUID valide qui n'existe pas dans la liste des données. La méthode devrait retourner un 404 avec un message de **person not found**.

```
i.  curl -X DELETE -i localhost:5000/person/11111111-589a-43b6-9a5d-d1601cf51111
```

Copied!Wrap Toggled!Executed!

Vous devriez voir une réponse JSON avec un code HTTP de 404 et un message de **person not found**.

```
i.   HTTP/1.1 404 NOT FOUND
ii.  Server: Werkzeug/2.2.2 Python/3.7.16
iii. Date: Sam, 31 Déc 2022 23:05:43 GMT
iv.  Content-Type: application/json
v.   Content-Length: 36
vi.  Connection: close
vii.
viii. {
ix.   "message": "person not found"
```

```
}
```

Copied!Wrap Toggled!

Solution

Vérifiez que votre travail correspond à la solution suivante.

Cliquez ici pour la réponse.

```
@app.route("/count")
def count():
    try:
        # Tentez de retourner le compte d'éléments dans 'data' sous forme de réponse JSON
        return {"data count": len(data)}, 200
    except NameError:
        # Gérez le cas où 'data' n'est pas défini
        # Retournez une réponse JSON avec un message et un code de statut 500 Internal Server Error
        return {"message": "data not defined"}, 500

@app.route("/person/<uuid:id>")
def find_by_uuid(id):
    # Itérez à travers la liste 'data' pour rechercher une personne avec un ID correspondant
    for person in data:
        # Vérifiez si le champ 'id' de la personne correspond au paramètre 'id'
        if person["id"] == str(id):
            # Retournez la personne correspondante en tant que réponse JSON avec un code de statut 200 OK
            return person
    # Si aucune personne correspondante n'est trouvée, retournez une réponse JSON avec un message et un code de statut 404 Not Found
    return {"message": "person not found"}, 404

@app.route("/person/<uuid:id>", methods=['DELETE'])
def delete_by_uuid(id):
    # Itérez à travers la liste 'data' pour rechercher une personne avec un ID correspondant
    for person in data:
        # Vérifiez si le champ 'id' de la personne correspond au paramètre 'id'
        if person["id"] == str(id):
            # Supprimez la personne de la liste 'data'
            data.remove(person)
```

```
xx.             # Retournez une réponse JSON avec un message confirmant la suppression et un code de statut 200 OK
xxi.            return {"message": f"Person with ID {id} deleted"}, 200
xxii.       # Si aucune personne correspondante n'est trouvée, retournez une réponse JSON avec un message et un code de statut 404 Not Found
xxiii.      return {"message": "person not found"}, 404
```

Copied!Wrap Toggled!

Étape 4 : Analyser le JSON du corps de la requête

Créons une autre API RESTful. Le client peut envoyer une requête `POST` à `http://localhost:5000/person` avec le JSON des détails de la personne dans le corps. Le serveur doit analyser la requête pour le corps et ensuite créer une nouvelle personne avec ces détails. Dans votre cas, pour créer la personne, il suffit d'ajouter à la liste `data`.

Vos Tâches

Créez une méthode appelée `add_by_uuid` avec le décorateur `@app.route`. Cette méthode doit être appelée lorsqu'un client fait une demande avec la méthode `POST` pour l'URL `/person`. La méthode n'acceptera aucun paramètre mais récupérera les détails de la personne à partir du corps JSON de la requête POST. La méthode retourne :

- L'`id` de la personne avec un code d'état HTTP 200 si la personne est ajoutée avec succès à la liste `data`.
- Un message `"Paramètre d'entrée invalide"` avec un code d'état HTTP 422 si le corps JSON est manquant ou vide.

Indice

Assurez-vous d'importer le module `request` de Flask. Vous l'utiliserez pour obtenir le prénom de la requête HTTP.

```
1. from flask import request
```

Copied!Wrap Toggled!

Vous pouvez utiliser le code suivant comme point de départ. Dans le code de production, vous ajouterez une logique pour valider le JSON entrant. Vous ne voudriez pas stocker des données aléatoires provenant d'un client. Vous pouvez omettre cette validation pour votre cas d'utilisation simple.

Cliquez ici pour un indice.

```
1. @app.route("/person", methods=['POST'])
2. def create_person():
3.     # Get the JSON data from the incoming request
4.     new_person = request.get_json()
5.
6.     # Check if the JSON data is empty or None
7.     if not new_person:
8.         # Return a JSON response indicating that the request data is invalid
9.         # with a status code of 422 (Unprocessable Entity)
10.        return {"message": "Invalid input, no data provided"}, 422
11.
12.    # Proceed with further processing of 'new_person', such as adding it to a database
13.    # or validating its contents before saving it
14.
15.    # Assuming the processing is successful, return a success message with status code 200
(Created)
16.    return {"message": "Person created successfully"}, 200
```

Copied!Wrap Toggled!

Saisies à faire dans le fichier server.py

```
@app.route("/person", methods=['POST'])
def add_by_uuid():
new_person = request.json
if not new_person:
return {"message": "Invalid input parameter"}, 422
# code to validate new_person ommited
try:
data.append(new_person)
except NameError:
return {"message": "data not defined"}, 500

return {"message": f"{new_person['id']}"}, 200
```

Vous pouvez tester le point de terminaison avec la commande CURL suivante. Assurez-vous que le serveur fonctionne dans le terminal comme dans les étapes précédentes.

```
curl -X POST -i -w '\n' \
  --url http://localhost:5000/person \
  --header 'Content-Type: application/json' \
  --data '{
        "id": "4e1e61b4-8a27-11ed-a1eb-0242ac120002",
        "first_name": "John",
        "last_name": "Horne",
        "graduation_year": 2001,
        "address": "1 hill drive",
        "city": "Atlanta",
        "zip": "30339",
        "country": "United States",
        "avatar": "http://dummyimage.com/139x100.png/cc0000/ffffff"
}'
```

Copied!Wrap Toggled!Executed!

Vous devriez voir une sortie similaire à celle donnée ci-dessous. Notez le statut de `200`, le type de contenu de `application/json`, et la sortie JSON de la personne avec le prénom **Abdel** :

.

```
HTTP/1.1 200 OK
Server: Werkzeug/2.2.2 Python/3.7.16
Date: Sun, 01 Jan 2023 23:14:34 GMT
Content-Type: application/json
Content-Length: 56
Connection: close

{
  "message": "4e1e61b4-8a27-11ed-a1eb-0242ac120002"
}
```

Copied!Wrap Toggled!

Vous pouvez également tester le cas où vous envoyez un JSON vide à l'endpoint en utilisant la commande suivante :

```
curl -X POST -i -w '\n' \
  --url http://localhost:5000/person \
  --header 'Content-Type: application/json' \
  --data '{}'
```

Copied!Wrap Toggled!Executed!

Le serveur doit renvoyer un code 422 avec un message de `Invalid input parameter`.

```
HTTP/1.1 422 UNPROCESSABLE ENTITY
Server: Werkzeug/2.2.2 Python/3.7.16
Date: Sun, 01 Jan 2023 23:15:54 GMT
Content-Type: application/json
Content-Length: 43
Connection: close

{
  "message": "Invalid input parameter"
}
```

Copied!Wrap Toggled!

Solution

Vérifiez que votre travail correspond à la solution suivante. Il existe plusieurs façons de mettre en œuvre cette solution. Notez que vous devez également vérifier si la liste `data` existe dans la solution et retourner un 500 si ce n'est pas le cas.

Cliquez ici pour la réponse.

```
@app.route("/person", methods=['POST'])
def add_by_uuid():
    new_person = request.json
    if not new_person:
        return {"message": "Invalid input parameter"}, 422
    # code to validate new_person ommited
    try:
        data.append(new_person)
    except NameError:
        return {"message": "data not defined"}, 500

    return {"message": f"{new_person['id']}"}, 200
```

Copied!Wrap Toggled!

Étape 5 : Ajouter des gestionnaires d'erreurs

Dans cette dernière partie du laboratoire, vous allez **ajouter des gestionnaires globaux au niveau de l'application pour traiter des erreurs comme 404 (non trouvé) et 500 (erreur interne du serveur)**. Rappelez-vous de la vidéo où Flask facilite la gestion de ces gestionnaires d'erreurs globaux en utilisant le décorateur errorhandler().

Si vous faites une requête invalide au serveur maintenant, Flask renverra une page HTML avec l'erreur 404. Quelque chose comme ceci :

Commande :

```
curl -X POST -i -w '\n' http://localhost:5000/notvalid
```

Copied!Wrap Toggled!Executed!

Je suis désolé, mais il semble que vous n'ayez pas fourni de texte à traduire. Veuillez soumettre le texte que vous souhaitez que je traduise.

```
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/2.2.2 Python/3.7.16
Date: Sun, 01 Jan 2023 23:21:54 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 207
Connection: close

<!doctype html>
<html lang=en>
<title>404 Not Found</title>
<h1>Not Found</h1>
<p>The requested URL was not found on the server. If you entered the URL manually, please check your spelling and try again.</p>
```

Copied!Wrap Toggled!

C'est génial, mais vous souhaitez renvoyer une réponse JSON pour toutes les demandes invalides.

Vos Tâches

Créez une méthode appelée `api_not_found` avec le décorateur `@app.errorhandler`. Cette méthode renverra un message de `API non trouvée` avec un code de statut HTTP de `404` chaque fois que le client demande une URL qui ne mène à aucun point de terminaison défini par le serveur.

Indice

Utilisez le décorateur `@app.errorhandler` et passez le code HTTP de `404`.

Vous pouvez utiliser le code suivant comme point de départ :

Cliquez ici pour un indice.

```
{insert errorhandler decorator here}({insert error code here})
def {insert method name here}(error):
    return {"message": "{insert error message here}"}, {insert error code here}
```

Copied!Wrap Toggled!

Saisie dans le fichier server.py

```
@app.errorhandler(404)
def api_not_found(error):
    # This function is a custom error handler for 404 Not Found errors
    # It is triggered whenever a 404 error occurs within the Flask application
    return {"message": "API not found"}, 404
```

Vous pouvez tester le point de terminaison avec la commande CURL suivante. Assurez-vous que le serveur fonctionne dans le terminal, comme dans les étapes précédentes.

```
curl -X POST -i -w '\n' http://localhost:5000/notvalid
```

Copied!Wrap Toggled!Executed!

Vous devriez voir une sortie similaire à celle ci-dessous. Notez le statut de `404`, le type de contenu de `application/json`, et le message de sortie JSON de **API non trouvée** :

```
HTTP/1.1 404 NOT FOUND
Server: Werkzeug/2.2.2 Python/3.7.16
Date: Sun, 01 Jan 2023 23:25:35 GMT
Content-Type: application/json
Content-Length: 33
Connection: close

{
  "message": "API not found"
}
```

Copied!Wrap Toggled!

Notez que le serveur ne renvoie plus de HTML mais du JSON comme requis.

Solution

Vérifiez que votre travail correspond à la solution ci-dessous. Il y a plusieurs façons de mettre en œuvre cette solution.

Cliquez ici pour la réponse.

```
@app.errorhandler(404)
def api_not_found(error):
    # This function is a custom error handler for 404 Not Found errors
    # It is triggered whenever a 404 error occurs within the Flask application
    return {"message": "API not found"}, 404
```

Copied!Wrap Toggled!

De la même manière, vous pouvez ajouter un gestionnaire d'erreurs global pour `500 (erreur interne du serveur)` également.

Vous pouvez enregistrer un gestionnaire d'erreurs global dans Flask pour toutes les exceptions non gérées en utilisant :

```
@app.errorhandler(Exception)
```

```
def handle_exception(e):
    return {"message": str(e)}, 500
```

Saisie dans le fichier server.py

```
@app.errorhandler(Exception)
def handle_exception(e):
    return {"message": str(e)}, 500
```

Copied!Wrap Toggled!Executed!

Cela indique à Flask de capturer toute exception non gérée levée n'importe où dans votre application et de la diriger vers ce gestionnaire, renvoyant une réponse 500 Internal Server Error avec le message d'erreur.

Vous pouvez délibérément lever une exception pour tester le gestionnaire global. Ajoutez la route suivante avant vos gestionnaires d'erreurs :

```
@app.route("/test500")
def test500():
    raise Exception("Forced exception for testing")
```

Copied!Wrap Toggled!Executed!

Ensuite, exécutez cette commande curl dans votre terminal :

```
curl http://localhost:5000/test500
```

Copied!Wrap Toggled!Executed!

Saisie dans le fichier server.py

```
@app.route("/test500")
def test500():
    raise Exception("Forced exception for testing")
```

Vous devriez voir une réponse comme :

.

```
{
  "message": "Forced exception for testing"
}
```

Copied!Wrap Toggled!Executed!

Auteur(s)

CF

© IBM Corporation 2023. Tous droits réservés.